DATA STRUCTURE AND ALGORITHM (SECJ2013)

SECTION 02

GROUP PROJECT

ANALYSIS OF SORTING ALGORITHMS

DUE DATE

18/11/2024

LECTURER

DR. ZURAINI BINTI ALI SHAH

PREPARED BY: GROUP 4

Name	No. Matric
GOE JIE YING	A23CS0224
LUQMAN HARITH BIN MOHD SHAHAR	A23CS3009
BASEL A. A. ABUNADA	A22EC4006
LUVINESH A/L SUDESH	A21EC0198

Analysis Report

Compare the performance (number of comparisons and swaps) of each algorithm.

In terms of comparisons, **3 algorithms** which are the **bubble sort, improved bubble sort and selection sort** have the **same number of comparisons which is 10**. In contrast, **Insertion sort** only has to make **9 total comparisons**. This suggests that insertion Sort is slightly more efficient in terms of comparisons. When an element is inserted, it only compares the element with the sorted region of the array until it finds the right position.

In terms of swaps, **the bubble sort, improved bubble sort and insertion sort** have the same number of swaps which is **6**. **Selection sort** swap only **2 times**. Selection Sort is more efficient in terms of swaps because it only swaps when it finds the minimum element in the unsorted portion, performing a swap only once per pass.

Discuss the efficiency of each algorithm based on your findings.

Sorting Algorithm	Efficiency	Explanation
Bubble Sort	$O(n^2)$	Bubble Sort alternates adjacent elements if they are out of order after continuously comparing them. Its quadratic temporal complexity makes it ineffective for big datasets. In the best case, when the array is already sorted, it can be optimized to O(n), but it performs best with small datasets.
Improved Bubble Sort	$O(n^2)$	Improved Bubble Sort is similar to Bubble Sort. It compares adjacent elements in the list and exchanges them if they are out of order. However, a boolean value is introduced in the algorithm to enhance the efficiency of Bubble Sort. This value signals whether an exchange of elements occurs during a particular pass. If the value is false, another pass will continue. The process stops, and no further passes are made if the boolean value remains true, indicating that the list is fully sorted. In the worst case $O(n^2)$, Improved Bubble Sort performs all passes and in the best case O(n), when the list is already sorted, the algorithm stops early after one pass. This minimizes unnecessary passes on sorted data.

Selection Sort	$O(n^2)$	Selection Sort works by repeatedly finding the smallest or largest element in an unsorted array and places it in its correct position. This is achieved by swapping the smallest element with the first unsorted element after each pass. The Selection Sort performs (n-1) passes where n is the number of elements in an array. The code provided has a nested for loop, which confirms that it runs in $O(n^2)$
Insertion Sort	$O(n^2)$	Insertion Sort works by taking one element at a time and placing it in the correct spot within the sorted part of the list. It's simple and works great for small or nearly sorted datasets. However, because it has to compare and shift elements for every insertion, it gets slow with larger datasets, taking $O(n^2)$ time in the worst case. If the list is already sorted, it's much faster, only needing O(n) time since no shifting is required.

Table 1: Explanation of each sorting algorithm including their big O notations

Include a table summarizing your results (e.g., comparisons and swaps for each algorithm).

	Data: [75, 95, 60, 88, 70]	
Sorting Algorithm	**Comparisons**	**Swaps**
Bubble Sort	10	6
Improved Bubble Sort	10	6
Selection Sort	10	2
Insertion Sort	9	6

Table 2: Comparisons and Swaps for each Algorithm

C++ Code Structure for Each Solution

<u>Bubble Sort</u>

```cpp
/*
Bubble Sort
Assignment 1
Name:   Basel A. A. Abunada
Matric: A22EC4006
*/

#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n, int &comparisons, int &swaps) {
    // Initialize counters for comparisons and swaps
    comparisons = 0;
    swaps = 0;

    // Outer loop runs for (n - 1) passes
    for (int i = 0; i < n - 1; i++) {
        // Inner loop for comparing adjacent elements
        for (int j = 0; j < n - i - 1; j++) {
            comparisons++; // record comparison
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swaps++; // record swap
            }
        }
    }
}
```

Figure 1:: Implementation of Bubble Sort with comments to indicate the comparisons and swapping

```
int main() {
    int marks[] = {75, 95, 60, 88, 70};
    int n = sizeof(marks) / sizeof(marks[0]);

    int comparisons, swaps;

    bubbleSort(marks, n, comparisons, swaps);

    // Display the sorted array
    cout << "Sorted Marks: ";
    for (int i = 0; i < n; i++) {
        cout << marks[i] << " ";
    }
    cout << endl;

    // Display the total number of comparisons and swaps
    cout << "Total Comparisons: " << comparisons << endl;
    cout << "Total Swaps: " << swaps << endl;

    return 0;
}
```

Figure 1.1:: Main driver code for Bubble Sort

Improved Bubble Sort

```cpp
/*
Improved Bubble Sort
Assignment 1
Name:   Goe Jie Ying
Matric: A23CS0224
*/

#include <iostream>
using namespace std;

void bubbleSort (int data[], int n)
{
    int temp;
    bool sorted = false; // false when swaps occur
    int comparisons = 0; // counter for comparisons
    int swaps = 0;       // counter for swaps

    for (int pass = 1; (pass < n) && !sorted; ++pass)
    {
        sorted = true; // assume sorted
        cout << "\nPass " << pass << endl;

        for (int x = 0; x < n-pass; ++x)
        {
            comparisons++;

            if (data[x] > data[x+1])
            { // exchange items
                temp = data[x];
                data[x] = data[x+1];
                data[x+1] = temp;
                sorted = false; // signal exchange
                swaps++;
                cout << "Swap at index [" << x << "]" << endl;
            } // end if
        } // end for

        // Display array after each pass
        cout << "Array after pass " << pass << ": ";
        for (int i = 0; i < n; i++)
        {
            cout << data[i] << " ";
        }
        cout << endl;

    } // end for

    cout << "\nTotal comparisons: " << comparisons << endl;
    cout << "Total swaps: " << swaps << endl;

} // end bubbleSort
```

Figure 2: Implementation of Improved Bubble Sort with comments to indicate the comparisons and swapping

```
} // end bubbleSort

int main()
{
    int data[] = {75, 95, 60, 88, 70};
    int n = 5;

    // Display the original array
    cout << "Original array: ";
    for (int i = 0; i < n; i++)
        cout << data[i] << " ";
    cout << endl;

    bubbleSort(data, n); // call the bubble sort function

    cout << endl;

    // Display the sorted array
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << data[i] << " ";
    cout << endl << endl;

    return 0;
}
```

Figure 2.1: Main driver code for Improved Bubble Sort

Selection Sort

```
/*
Selection Sort
Assignment 1
Name:   LUQMAN HARITH BIN MOHD SHAHAR
Matric: A23CS3009
*/

#include <iostream>
using namespace std;

void selectionSort(int arr[], int n, int &comparisonCount, int &swapCount)
{
    comparisonCount = 0;
    swapCount = 0;

    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            comparisonCount++; // Count the comparison
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }
        if (minIndex != i)
        {
            swap(arr[i], arr[minIndex]); // Swap elements
            swapCount++;                 // Count the swap
        }
    }
}
```

Figure 3: Implementation of Selection Sort with comments to indicate the comparisons and swapping

```
int main()
{
    int arr[] = {75, 95, 60, 88, 70};
    int n = sizeof(arr) / sizeof(arr[0]);
    int comparisonCount, swapCount;

    // Print unsorted array
    cout << "Unsorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    selectionSort(arr, n, comparisonCount, swapCount);

    // Print sorted array
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Print metrics
    cout << "Total comparisons: " << comparisonCount << endl;
    cout << "Total swaps: " << swapCount << endl;

    return 0;
}
```

Figure 3.1: Main driver code for Selection Sort

Insertion Sort

```
/*
Insertion Sort
Assignment 1
Name:   LUVINESH A/L SUDESH
Matric: A21EC0198
*/

#include <iostream>
#include <vector>
using namespace std;


void insertionSort(vector<int>& arr, int& comparisons, int& swaps) {
    int n = arr.size(); // Get the size of the array
    comparisons = 0;
    swaps = 0;

    for (int i = 1; i < n; ++i) {
        int key = arr[i]; // Element to be placed at the correct position
        int j = i - 1; // Index to compare with the sorted part of the array

            while (j >= 0 && arr[j] > key) {
            ++comparisons; // counter for comparisons
            arr[j + 1] = arr[j]; // Shift element to the right
            --j; // Move to the next element
            ++swaps; // Count a shift (swap)
        }

        arr[j + 1] = key; // Place the 'key' at the correct position
        if (j >= 0) ++comparisons;
    }
}
```

Figure 4: Implementation of Insertion Sort with comments to indicate the comparisons and swapping

```
// Function to display the elements of an array
void displayArray(const vector<int>& arr) {
    for (int num : arr) {
        cout << num << " ";
    }
    cout << endl;
}

int main() {

    vector<int> marks = {75, 95, 60, 88, 70};

    int comparisons = 0, swaps = 0;

    // Display the original array
    cout << "Original Array: ";
    displayArray(marks);

    // Perform Insertion Sort
    insertionSort(marks, comparisons, swaps);

    // Display the sorted array
    cout << "Sorted Array: ";
    displayArray(marks);

    // Output the number of comparisons and swaps
    cout << "Total Comparisons: " << comparisons << endl;
    cout << "Total Swaps: " << swaps << endl;

    return 0;
}
```

Figure 4.1: Main driver code for Insertion Sort

Reflection:

Luvinesh Sudesh:

Insertion sort is a simple and easy-to-understand algorithm that works like sorting playing cards in your hand—it picks one item at a time and places it in the right spot. It's great for small or almost-sorted datasets because it's quick and doesn't need much extra work. The algorithm is reliable and keeps the order of equal items, but it gets slow with larger datasets because it has to do a lot of comparisons and swaps. Learning about insertion sort is a good reminder that even basic algorithms can be useful and teach us important lessons about efficiency and problem-solving.

Goe Jie Ying:

After completing this assignment, I learned more about Bubble Sort, Improved Bubble Sort, Selection Sort, and Insertion Sort algorithms, as well as their differences when applied to the same dataset. By analyzing the number of comparisons and swaps, I was able to observe the strengths and weaknesses of each method. I believe that this assignment will be highly beneficial for the upcoming test. Besides collaborating with my teammates throughout the assignment, I gained team working and communication skills, which are essential for future group projects and professional settings..

Luqman Harith bin Mohd Shahar:

Completion of this assignment has reinforced my knowledge related to selection sort. What surprised me was that even if the array has been sorted, the selection sort will still perform comparisons due to the program running in (n-1) operations. On top of that, I was able to manually see through my drawing on how the elements of the array are sorted. In terms of teamwork, this assignment taught me to be a teamplayer and use my communication skills to elaborate on what needs to be done. The team has been cooperative.

Basel A. A. Abunada:

It was a great experience learning to work on Bubble Sort. It's a straightforward process, and by comparing and swapping elements one after the other, writing the code helped me comprehend how it operates. I found that it performs a lot of pointless work, which makes it inefficient, particularly when dealing with larger datasets. Nevertheless, it is simple to comprehend and a good place to start when understanding sorting algorithms. Collaborating with my group was beneficial as well because it allowed me to discuss ideas with my teammates and observe how various algorithms compare.

No.: Date:

Bubble Sort

marks array = {75, 95, 60, 88, 70}

Pass 1:-

	0	1	2	3
[4] →	70	70	70	95
[3] →	88	88	95	70
[2] →	60	95	88	88
[1] →	95	60	60	60
[0] →	75	75	75	75
Sorted:	F	F	F	F

index 1: we swapped 95 ↔ 60
index 2: we swapped 95 ↔ 88
index 3: we swapped 95 ↔ 70

Comparisons: 4 , Swaps: 3

Pass 2:-

	0	1	2
[4] →	95	95	95
[3] →	70	70	88
[2] →	88	88	70
[1] →	75	75	75
[0] →	60	60	60
Sorted:	F	F	F

index 0: we swapped 75 ↔ 60
index 2: we swapped 88 ↔ 70

Comparisons: 3 , Swaps: 2

Scanned with CamScanner

No.: Date:

Pass 3 :-

	0	1
[4]→	95	95
[3]→	88	88
[2]→	70	75
[1]→	75	70
[0]→	60	60
Sorted:-	F	T

index 1 : we swapped 75 ↔ 70

Comparisons: 2 , swaps: 1

Pass 4 :-

	0
[4]→	95
[3]→	88
[2]→	75
[1]→	70
[0]→	60
Sorted :	T

There's no swapping.

Comparisons: 1

Summary :-

- during the 1st Pass we did 4 comparisons and 3 swaps, in the 2nd we did 3 comparisons and 2 swaps, the 3rd Pass contains 2 comparisons and 1 swap lastly the 4th Pass 1 comparison and no swapping.

Total comparison : 10
Total swaps : 6

#

Improved Bubble Sort

5 student marks array = {75, 95, 60, 88, 70}

Pass 1

x=	0	1	2	3	4
[4]	70	70	70	70	95
[3]	88	88	88	95	70
[2]	60	60	95	88	88
[1]	95	95	60	60	60
[0]	75	75	75	75	75
Sorted=	T	F	F	F	

No. Comparison = 4
No. Swaps = 3

Pass 2

x=	0	1	2	3
[4]	95	95	95	95
[3]	70	70	70	88
[2]	88	88	88	70
[1]	60	75	75	75
[0]	75	60	60	60
Sorted:	F	F	F	

No. Comparisons = 3
No. Swaps = 2

Pass 3

x=	0	1	2
[4]	95	95	95
[3]	88	88	88
[2]	70	70	75
[1]	75	75	70
[0]	60	60	60
Sorted:	T	F	

No. Comparisons = 2
No. Swaps = 1

Pass 4

x=	0	1
[4]	95	95
[3]	88	88
[2]	75	75
[1]	70	70
[0]	60	60
Sorted:	T	

No. Comparisons = 1
No. Swaps = 0

Total comparison = 10
Total swaps = 6

NO : ..... DATE : .....

Selection Sort

5 student marks array = [75, 95, 60, 88, 70]

First Pass

Index	Array	
0	75	60
1	95	95
2	60	75
3	88	88
4	70	70

Steps:

- Compare 75 (index 0) with 95 (index 1). No swap
- Compare 75 with 60. 60 is the new minimum
- Compare 60 with 88, no change
- Compare 60 with 70, no change

*Total comparisons = 4, *Total swap = 1

Second Pass

Index	Array	
0	60	60
1	95	70
2	75	75
3	88	88
4	70	95

Steps:

- Compare 95 with 75. 75 is the new minimum
- Compare 75 with 88, no change
- Compare 75 with 70, 70 is the new minimum
- Swap 95 and 75 (index 1 and 4)

*Total comparisons = 3, *Total swap = 1

Third Pass

Index	Array	
0	60	60
1	70	70
2	75	75
3	88	88
4	95	95

Steps:

- Compare 75 with 88, no change
- Compare 75 with 95, no change

*Total comparisons = 2, *Total swap = 0

Scanned with CamScanner

NO : ..... DATE : .....

Fourth Pass

Index	Array		
0	60		60
1	70		70
2	75		75
3	88		88
4	95		95

Steps :

- Compare 88 with 95, no change

* Total comparisons = 1,
* Total swap = 0

Final Array

[60, 70, 75, 88, 95]

* Total number of comparisons = 4 + 3 + 2 + 1 = 10
* Total number of swaps = 1 + 1 + 0 + 0 = 2

Scanned with CamScanner

ASSIGNMENT 1 - INSERTION SORT

75 95 60 88 70 → 75 95 60 88 70

→ 75 95 60 88 70

→ 75 60 95 88 70 } 1

→ 60 75 95 88 70

→ 60 75 88 95 70

→ 60 75 88 70 95

→ 60 75 70 88 95

→ 60 70 75 88 95

→ 60 70 75 88 95

Total swap = 6

Total comparisons = 9